Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Защита веб-периметра и ограничение интенсивности сетевого трафика средствами межсетевых экранов прикладного уровня»

Выполнил:

Студент 4 курса

Группы ИИ-26

Кушнеревич Е.А.

Проверила:

Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx.

Ход работы:

Вариант	WAF: Блокируемый User-Agent	ID правила WAF	Rate Limit: Эндпоинт (Target)	Rate Limit: rate	Rate Limit: burst
8	`Python-urllib/3.8`	10008	`/api/register`	`2r/s`	`4`

Задание 1. Базовая фильтрация угроз через WAF (OWASP Core Rule Set)



Задание 2. Написание пользовательских правил фильтрации (L7)





```
vboxuser@Ubany:~/Downloads$ curl -i -A Mozilla/5.0 http://127.0.0.1/
HTTP/1.1 200 OK
Server: nginx/1.30.6
Date: Wed, 11 Nov 2026 11:48:05 GMT
Content-Type: application/json
Content-Length: 38
Connection: keep-alive

{"status": "ok", "hint": "try /?id=1"}vboxuser@Ubany:~/Downloads$ sudo docker compose logs waf | grep 10008
WARN[0000] /home/vboxuser/Downloads/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential c
onfusion
waf-1  | 2026/11/11 11:47:30 [error] 577#577: *4 [client 172.18.0.1] ModSecurity: Access denied with code 403 (phase 1). Matched "Operator `Contains'
with parameter `python-urllib/3.8' against variable `REQUEST_HEADERS:User-Agent' (Value: `Python-urllib/3.8' ) [file "/etc/modsecurity.d/owasp-crs/rul
es/custom.conf"] [line "2"] [id "10008"] [rev ""] [msg "Blocked scanner User-Agent Python-urllib/3.8"] [data ""] [severity "0"] [ver ""] [maturity "0"
] [accuracy "0"] [tag "modsecurity"] [hostname "127.0.0.1"] [uri "/"] [unique_id "179439765093.476675"] [ref "o0,17v43,17t:lowercase"], client: 172.18
.0.1, server: _, request: "GET / HTTP/1.1", host: "127.0.0.1"
waf-1  | {"transaction":{"client_ip":"172.18.0.1","time_stamp":"Wed Nov 11 11:47:30 2026","server_id":"7d51bc790c5dc2864fe7cb4f12ae86d06ab3cbd8","clie
nt_port":49542,"host_ip":"172.18.0.3","host_port":80,"unique_id":"179439765093.476675","is_interrupted":true,"request":{"method":"GET","http_version":
"1.1","hostname":"127.0.0.1","uri":"/","headers":{"Host":"127.0.0.1","User-Agent":"Python-urllib/3.8","Accept":"*/*"}},"response":{"body":"","http_cod
e":403,"headers":{}},"producer":{"modsecurity":"ModSecurity v3.0.17 (Linux)","connector":"ModSecurity-nginx v1.0.4","secrules_engine":"Enabled","compo
nents":["OWASP_CRS/4.30.0\""]},"messages":[{"message":"Host header is a numeric IP address","details":{"match":"Matched \"Operator `Rx' with parameter
 `(?:^([\\d.]+|\\[[\\da-f:]+\\])|[\\da-f:]+)(:[\\d]+)?$)' against variable `REQUEST_HEADERS:Host' (Value: `127.0.0.1' )","reference":"o0,9o0,9v21,9","r
uleId":"920350","file":"/etc/modsecurity.d/owasp-crs/rules/REQUEST-920-PROTOCOL-ENFORCEMENT.conf","lineNumber":"756","data":"127.0.0.1","severity":"4"
,"ver":"OWASP_CRS/4.30.0","rev":"","tags":["modsecurity","application-multi","language-multi","platform-multi","attack-protocol","paranoia-level/1","O
WASP_CRS","OWASP_CRS/PROTOCOL-ENFORCEMENT","capec/1000/210/272"],"maturity":"0","accuracy":"0"}},{"message":"Blocked scanner User-Agent Python-urllib/
3.8","details":{"match":"Matched \"Operator `Contains' with parameter `python-urllib/3.8' against variable `REQUEST_HEADERS:User-Agent' (Value: `Pytho
n-urllib/3.8' )","reference":"o0,17v43,17t:lowercase","ruleId":"10008","file":"/etc/modsecurity.d/owasp-crs/rules/custom.conf","lineNumber":"2","data"
:"","severity":"0","ver":"","rev":"","tags":["modsecurity"],"maturity":"0","accuracy":"0"}}]}}
```

Задание 3. Защита от L7 DDoS-атак (Rate Limiting)

```
vboxuser@Ubany:~/Downloads$ cat nginx/user_nginx.conf
limit_req_zone $binary_remote_addr zone=register_limit:10m rate=2r/s;

server {
    listen 80 default_server;
    server_name _;

    location /api/register {
        limit_req zone=register_limit burst=4;
        limit_req_status 503;

        proxy_pass ${BACKEND};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location / {
        proxy_pass ${BACKEND};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

```
vboxuser@Ubany:~/Downloads$ ab -n 500 -c 10 http://127.0.0.1/api/register
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 127.0.0.1 (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Finished 500 requests


Server Software:        nginx/1.30.6
Server Hostname:        127.0.0.1
Server Port:            80

Document Path:          /api/register
Document Length:        45 bytes

Concurrency Level:      10
Time taken for tests:   3.505 seconds
Complete requests:      500
Failed requests:        492
   (Connect: 0, Receive: 0, Length: 492, Exceptions: 0)
Non-2xx responses:      492
Total transferred:      183100 bytes
HTML transferred:       97284 bytes
Requests per second:    142.63 [#/sec] (mean)
Time per request:       70.110 [ms] (mean)
Time per request:       7.011 [ms] (mean, across all concurrent requests)
Transfer rate:          51.01 [Kbytes/sec] received
```

```
Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.8      0       7
Processing:    13   45 195.0     20    2032
Waiting:       13   44 194.9     19    2032
Total:         13   45 195.1     20    2032

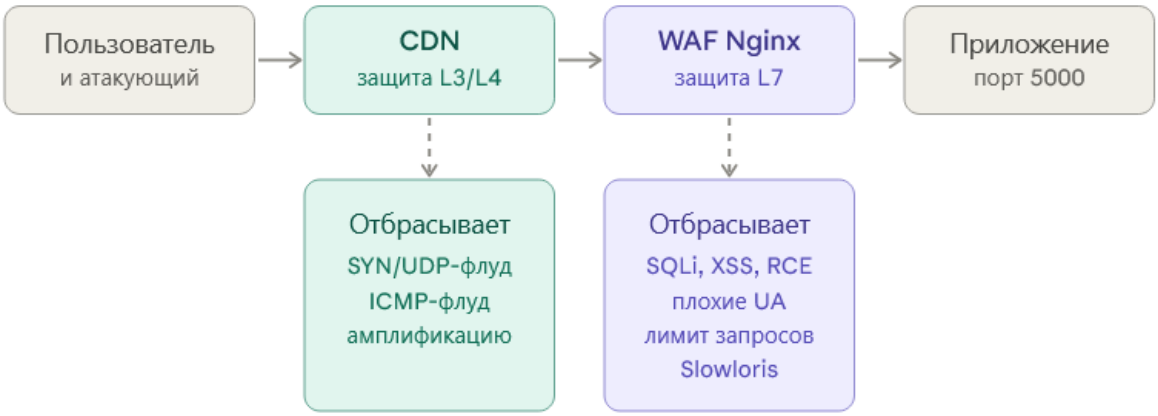
Percentage of the requests served within a certain time (ms)
  50%     20
  66%     21
  75%     24
  80%     26
  90%     37
  95%     47
  98%     70
  99%   1495
 100%   2032 (longest request)
```

Задание 4. Проектирование профилей защиты (Без написания кода)

```
client_header_timeout 10s;
client_body_timeout   10s;
keepalive_timeout     15s;
send_timeout          10s;
reset_timedout_connection on;
client_header_buffer_size   1k;
large_client_header_buffers 4 8k;
client_max_body_size        1m;
```

client_header_timeout 10s. - по умолчанию в nginx 60 секунд, и этим пользуется атакующий. Нормальный клиент отправляет заголовки (до нескольких КБ) за доли секунды, даже в мобильной сети это 1–2 секунды. 10 секунд дают запас в 5 раз и при этом обрывают медленного атакующего с ответом 408.
client_body_timeout 10s. - таймер считается между двумя последовательными чтениями, а не на всё тело. Живой клиент присылает данные непрерывно, а Slowloris-подобный POST делает паузы, и соединение закрывается.
keepalive_timeout 15s. - браузер забирает ресурсы страницы пачкой за несколько секунд, поэтому 15 секунд хватает для повторного использования соединения. Значение по умолчанию 75 секунд слишком долго занимает слоты впустую.
Дополнительно. Ограничение limit_conn по IP и малые буферы не дают одному источнику занять много слотов worker_connections. При 10-секундных таймаутах атакующему приходится постоянно переоткрывать соединения, и он упирается в лимит.

Схема:



CDN (уровни L3/L4) - весь внешний трафик сначала попадает на распределённую сеть CDN. Она работает по технологии Anycast, поэтому поток атаки делится между десятками узлов по всему миру. Пропускная способность CDN на порядки больше, чем у одного сервера, и она поглощает SYN-, UDP- и ICMP-флуд, а также атаки с амплификацией (DNS, NTP). Статический контент отдаётся из кэша, поэтому он не нагружает origin. Реальный IP сервера скрыт за CDN.
WAF Nginx + ModSecurity (уровень L7) - до WAF доходит уже очищенный от объёмных атак трафик. Здесь анализируется содержимое HTTP: правила OWASP CRS ловят SQLi, XSS и RCE, пользовательское правило 10008 блокирует сканер Python-urllib/3.8, а limit_req (2r/s, burst 4) и таймауты ограничивают L7 DDoS, брутфорс и Slowloris.
Сервер приложения - он получает только легитимные запросы. Файрвол origin пропускает трафик лишь с адресов CDN. Иначе атакующий узнает реальный IP и пойдёт в обход CDN и WAF.

Вывод: Проделав данную лабораторную работу я освоил методов развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx.